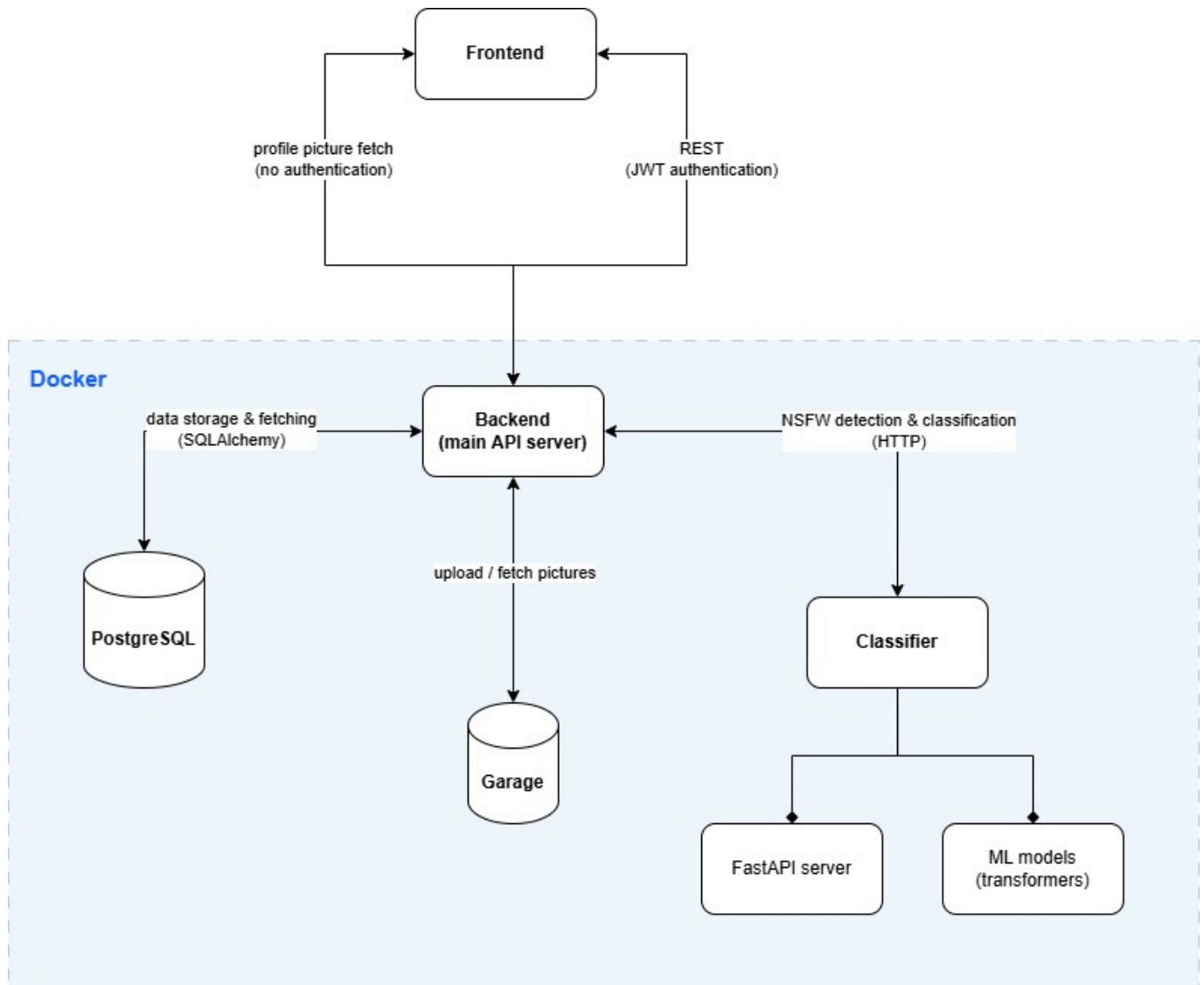


Overview

This document outlines the main design choices behind this project, as well as various security and safety considerations.



Database

SQL vs NoSQL

An SQL database, specifically PostgreSQL, was chosen for this project given the simple and clear structure of the data models (user & userProfile). The alternative would have been a NoSQL solution, such as MongoDB, which can facilitate certain parts of the process. For example, given the JSON format of the backend's responses, the same JSON format of MongoDB documents could be re-used.

However, owing to the small scale of this project, PostgreSQL was favored to focus on a light and performant database solution.

ORM

Rather than manually written SQL statements, an ORM was used to model the different tables and their columns, namely SQLAlchemy.

These can be found under `models/`. Note that the schemas found under `schemas/` are only used for communication between the frontend and backend - they do not correspond to the actual table designs.

The database session is centrally managed via `database/session.py`, which creates a local engine session.

Finally, all data models inherit from the class `Base(DeclarativeBase)` in `database/base.py`. This ensures the different data model classes define their table names using `__tablename__`, which will subsequently be used via `Base.metadata` to initialize the tables.

Foreign Key

One consideration to note is the relationship between a user and their user profile.

Given that a single user can only have a single user profile, a one-to-one relationship was created.

This was achieved by using the primary key of the users table (`id`) as a foreign and primary key for the user_profiles table (`user_id`).

Indexing

The primary keys of the two tables are indexed by default (users, user_profiles).

Additional indexing was performed for the various columns of the user_profiles tables, to optimize the filtering process.

Specifically, the following indices were created:

- `age`: B-tree index - this is to optimize the age range queries ($\text{min_age} < x < \text{max_age}$)
 - Notably, a B-tree index is useful, given that the values are indexed in a sorted manner, thus giving us an optimized approach to tree traversal for finding a specific range.
- `gender`: B-tree index - this is to optimize the exact matching based on the three fixed possible values (male, female, other)
- `country_of_origin`: B-tree index - same reasoning as for `gender`, given the fixed list of possible ISO country codes
- `place_of_residence`: GIN index with trigram ops - compared to `country_of_origin` and `gender`, we do not use exact matching for the filtering here. Rather, we need to support partial matching, given the free-form input from the user's side. For example, the search term "york" should retrieve a user profile with the value "New York" for its `place_of_residence` field.
 - In this case, a GIN index is favored for string-based columns where the values do not stem from a fixed list of potential values (e.g., `gender`), but where the values can differ greatly.
 - As for `trigram ops`, this allows us to break down a given string into trigrams, i.e., three-character sequences, which can for example facilitate searches in the case of minor typos.

API

Framework

Given the Python context of this project, a corresponding REST API server framework had to be chosen. To keep things short, though more mature frameworks such as **Django** exist, the newer **FastAPI** framework was chosen for this purpose owing to its more modern design.

Endpoints

The backend features different categories of endpoints:

1. **Authentication:** these endpoints do not require authentication, given that they serve as the primary entry point for the frontend to actually set up a user account and retrieve their JWT.

1. **/users/register** POST (Register): the username and password are passed as part of the request's JSON body.
 - The server responds with a JWT, as well as the user's username and role. Immediately responding with the JWT ensures the user does not have to manually login after having registered.
2. **/users/login** POST (Login): the username and password are passed as form data, which is the standard for OAuth2-based password authentication.
 - The server responds with a JWT, as well as the user's username and role.

2. User Profile

1. **/users/profile** PUT (create/update user profile): given that the id of the created user_profile is not server-generated, but actually based on the authenticated user's existing id, **PUT** is used as the HTTP method rather than **POST**. Similarly, as the same endpoint is used to update (replace entirely) an existing user_profile, **PUT** was favored over **PATCH** (partial updating).
 - Whereas the user profile details, such as age and description, are formatted as JSON, the profile picture is attached to the request as **multipart/form-data**.
 - However, given that a request cannot feature both a JSON body and form data, the profile details were also included in the form data, specifically as a JSON string field.
2. **/users/profile** GET (retrieve user profile of authenticated user): the server responds with the user's profile data in JSON format.
 - Should the user not have a user profile yet, a standard **404** HTTP status code is returned instead.
3. **/users/profile/picture/status** GET (poll the classification status of the profile picture): the user can poll this lightweight endpoint for their uploaded profile picture's classification status.
4. **/users/profile/picture/{filename}** GET (retrieve the profile picture's bytes): a dedicated endpoint is available for retrieving the byte data of a saved profile picture.
 - This endpoint is fully public and does not require authentication. Since **Garage** does not permit anonymous access, the backend proxies the image data to the frontend. Making this endpoint public avoids requiring the frontend to attach a JWT on every image request, e.g., for embedding profile pictures in a web page.

3. Admin (requires admin role in addition to a valid JWT)

1. **/admin/users/profile/{user_id}** GET (retrieve a specific user's profile by id).
2. **/admin/users/profiles** GET (retrieve all user profiles, with optional filtering): supports filtering by **min_age**, **max_age**, **exact_age**, **genders**, **place_of_residence**, and **country_of_origin**.

Storage

Storing large media, such as pictures and videos, is not efficient in a standard SQL table. Instead, a separate storage layer was used for this purpose.

Specifically, the **Garage** storage solution was used, as its S3 object store based design lends itself well to future deployment to a cloud solution, which similarly uses the same S3 object store, e.g., AWS.

Technically speaking, the storage layer is fairly simple in design, with a single **profile-pictures** bucket being created to store the different users' profile pictures. Upon storage, the picture's reference within that bucket is returned to the backend, which will store this reference as a column within the `user_profiles` table.

Given that **Garage** does not allow "anonymous" access by default, i.e., the frontend trying to query it for a picture would be met with a "forbidden access" response, the backend instead provides its own public and non-authenticated endpoint (`/users/profile/picture/{filename}`), which fetches the picture from **Garage** and forwards it to the frontend. The endpoint is intentionally unauthenticated so that profile pictures can be embedded and rendered freely by the frontend without attaching credentials to each image request.

More generally, other solutions exist, such as **MinIO**, though **Garage** was favored for this project given its lightweight nature.

Classification

A major sticking point in regard to performance and user experience is the process of classifying an uploaded profile picture:

1. We firstly have to verify if the picture contains NSFW content.
2. Secondly, if no NSFW content is detected, we want to classify the content of the picture, e.g., as "Tiger Cat".

For both purposes, relevant machine learning models were used via the **transformers** Python module, namely **Falconsai/nsfw_image_detection** and **google/vit-base-patch16-224**.

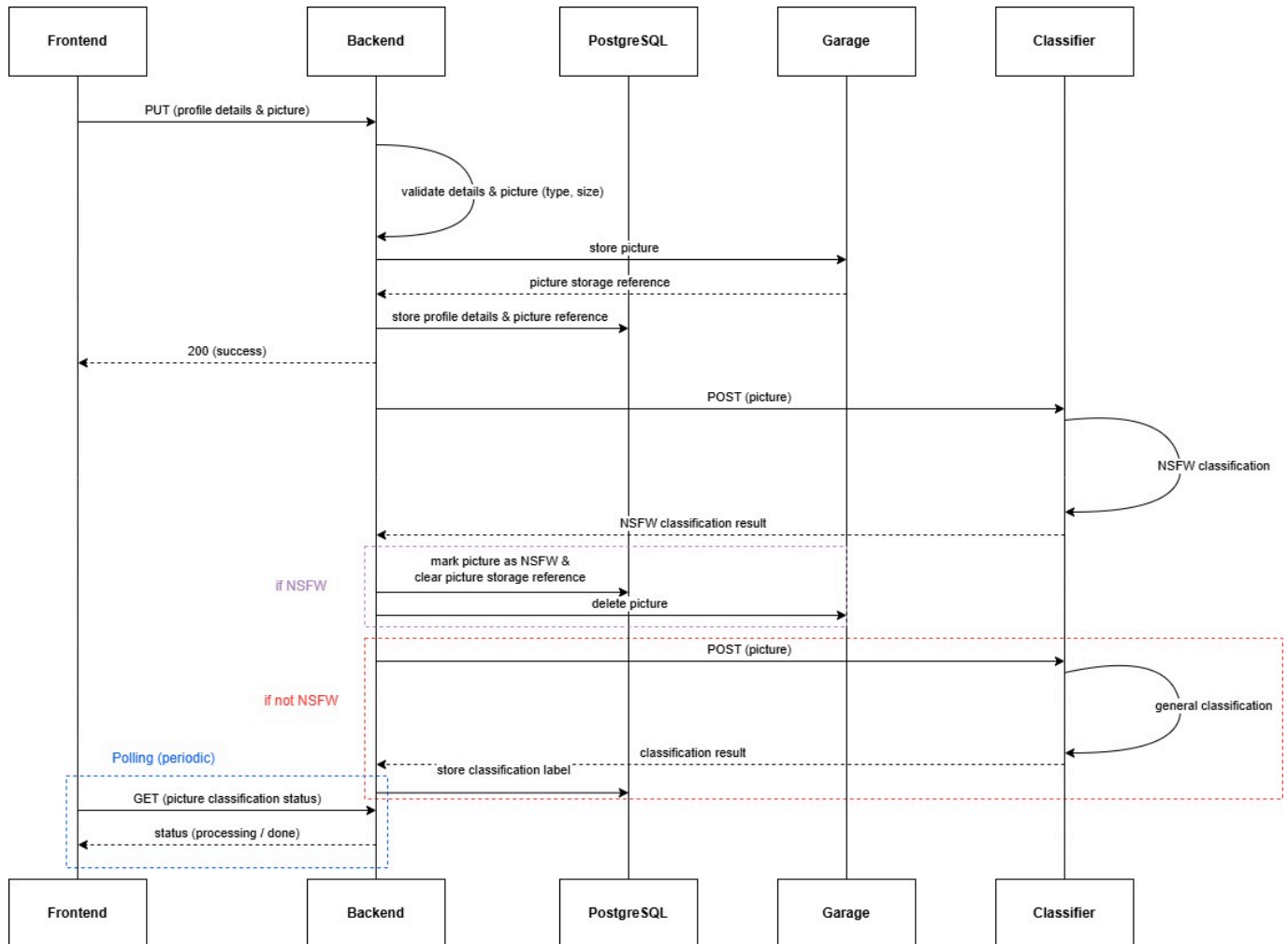
Given the separate business logic of this classification process, it was separated into its own service, rather than being bundled into the same FastAPI server.

More importantly, when connecting the two services together, an important design choice has to be made: upon uploading their profile details and picture to the FastAPI server, the backend (API server) will proceed with validating the profile details and picture, uploading the picture to Garage, and storing the profile details along with the picture reference in PostgreSQL — all synchronously before responding to the frontend.

However, the frontend should not have to wait a long time for a response from the API server, owing to this potentially expensive classification process.

As such, the API server delegates the classification to a background task, which only starts after the response has been returned to the frontend. The classification service is therefore never called as part of the main request cycle.

Subsequently, the frontend is provided with a separate polling endpoint (`/users/profile/picture/status`), which allows them to periodically check on their profile picture's classification result, until finally that background task has finished as well.



Filtering

As part of the admin dashboard, where the user profiles can be inspected, the admin also has access to various filtering options: `min_age`, `max_age`, `exact_age`, `genders`, `place_of_residence`, and `country_of_origin`.

The filtering logic could technically be done on the frontend-side, given the fairly small scale and simple logic of the entire process.

However, the filtering logic is kept on the backend side for scaling purposes, given that the database has indexes for the different columns (age, gender, etc.) to optimize the different database lookup queries. In a larger setting with thousands of database entries, the filtering logic would be much more efficient using these indexes, than the frontend variant of manually filtering the entire list of user profiles.

Docker

Services

To enable an easy deployment strategy for the entire backend, the various components have been set up as separate Docker services, namely:

- `backend`: the main FastAPI api server
- `postgres`: the PostgreSQL database
- `classification-service`: the separate FastAPI server hosting the ML models

- `garage`: the storage layer for the profile pictures

Secrets

Various secrets, such as the key used for signing a JWT, are stored as part of an environment file (`.env`), which is not committed to the git history or pushed to the remote GitHub repository.

In a production setting, these secrets would preferably be stored as Docker secrets, or perhaps GitHub secrets.

Sample

For testing purposes, a dedicated sample data generation script `sample-generator.py` is provided. This script generates 1 admin user, alongside 10 regular users. The `faker` module is used to create fake first and last names, as well as randomly picked countries of origin and fake places of residence.

This script is included as an additional docker service (`seeder`), which can be run using the command `docker-compose --profile seed run --rm seeder` (while the docker containers are running).

Validation

Data validation is performed on both the frontend and backend.

On the frontend, the client is given clear errors and warnings when editing their profile or creating their user account. On the backend, the same validations are included as part of the pydantic-based schemas.

The exact validations are the following:

- `username`:
 - `minLength`: 4
 - `maxLength`: 32
 - `pattern`: alphanumeric, as well as underscores and hyphens (allowed characters)
- `password`:
 - `minLength`: 8
 - `maxLength`: 64
- `first_name`:
 - `minLength`: 2
 - `maxLength`: 32
 - `pattern`: a-z characters and hyphens
- `last_name`:
 - `minLength`: 2
 - `maxLength`: 32
 - `pattern`: a-z characters and hyphens
- `age`:
 - `ge`: 18 (greater than or equal)
 - `le`: 120 (less than or equal)
- `place_of_residence`:
 - `minLength`: 2
 - `maxLength`: 100
- `country_of_origin`: country code in [ISO 3166-1 alpha-2](#) format.
- `description`:

- `maxLength`: 500

Additionally, the profile picture has the following constraints:

- Supported file types: JPG, PNG
- Maximum file size: 5MB

Security

The following security measures have been taken:

- **Backend:**
 - Password hashing: the user's password is not stored as plaintext in the users table, but its hashed version is stored instead.
 - Password check: the popular `pwdlib` module's `PasswordHash` class is used for both hashing and verifying hashes.
 - JWT expiration: a limited expiration time is attached to generated access tokens. (1 day)
 - Endpoint authentication: user-related endpoints are placed behind authentication interceptors, i.e., a JWT is expected to be attached to incoming requests, while admin-related endpoints perform an additional admin role check based on the token and database username/role lookup.
 - Timing attack prevention: the login endpoint always performs a password hash verification regardless of whether the username exists. This prevents an attacker from inferring whether a username is registered based on the server's response time.
- **Frontend:**
 - Guards: endpoint guards are used to prevent the user from accessing certain pages when they are not logged in. A similar guard is used to provide access to admin-related pages to admin users only.
 - Environment: separate environment files are used to store details such as the API endpoint for development and production builds.

CI/CD

Regarding the continuous integration and development processes, GitHub Actions are used to verify that the docker image builds correctly and that the services run as well.

These checks are done upon each push and pull request to the main branch.

A separate check is also performed to verify if the dependencies of the main Python project can be successfully installed via pip.

Finally, pre-commit linting is performed using [ruff](#).

Other Notes

Though not implemented, other things to consider for a production-ready deployment:

- Password strength: an algorithm could be used to calculate the strength of the user's chosen password during registration, while also forbidding them from choosing a password below a certain strength threshold.
 - Potential solution: [Zxcvbn](#)

- JWT storage: currently, the frontend stores their JWT upon authentication as part of the local storage. This is not secure, given it can be read via JavaScript, e.g., this opens it up to potential token theft via a XSS vulnerability.
 - Potential solution: have browser send the JWT via an HttpOnly cookie, which cannot be accessed via JavaScript.
- Refresh tokens: rather than solely using access tokens, the authentication system could benefit from refresh tokens as well.
 - Reason: this could notably support the invalidation of tokens server-side, e.g., in case of a security breach or account theft.
- Rate limiting: the login and registration endpoints are currently unthrottled. In production, rate limiting should be applied to prevent brute-force attacks.
 - Potential solution: a reverse proxy such as `nginx` or a FastAPI middleware (e.g., `slowapi`) can enforce request rate limits per IP.
- HTTPS/TLS: all traffic between the frontend, backend, and storage layer should be encrypted in transit. In production, this can typically be handled by a reverse proxy or a cloud load balancer.
- CORS: the backend should explicitly configure allowed origins to restrict which frontends can make cross-origin requests to the API, rather than allowing any origin ("*").
- Database migrations: the current setup uses `Base.metadata.create_all()` to initialise tables, which is suitable for development but does not handle schema changes on existing databases. In production, a migration tool such as `Alembic` could be used to manage incremental schema changes.
- Observability: logging, a health check endpoint and metrics collection would be necessary for monitoring the application in a production environment.
 - Currently, only a `/health` endpoint is implemented.